

Chapter 2.

Natural Language Processing

- | 1.1. What is NLP?
- | 1.2. Introduction to Search
- | 1.3. Processing text
- | 1.4. Representing text
- | 1.5. Similarity measures
- | 1.6. Next up

- Understand what NLP is and its real-world applications
- Explain why text is challenging for computers
- Perform text preprocessing (cleaning, tokenization, normalization)
- Implement keyword search (simple and TF-based)
- Start to understand sparse vs dense vector
- Implement similarity-based search using TF-IDF
- Cluster documents using machine learning (in order to visualize similarity)

NLP = Teaching computers to **understand, interpret, and generate human language**

 **Search engines:** Understanding what you really mean when you search, and finding it

 **Email filtering:** Identifying spam vs important messages

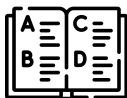
 **Chatbots:** Understanding and replying to customer questions

 **Document analysis:** Organizing and finding information in large document collections

 **Recommendation systems:** Finding similar content (movies, products, jobs)

... so much more

NLP Use cases



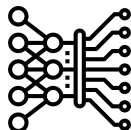
Representative
models



Retrievals



Text Classification



Generative
models



Text translation



Text generation

Let's start with a demo

Let's start with a demo

You're building a system to help recruiters search through thousands of CVs/resumes.

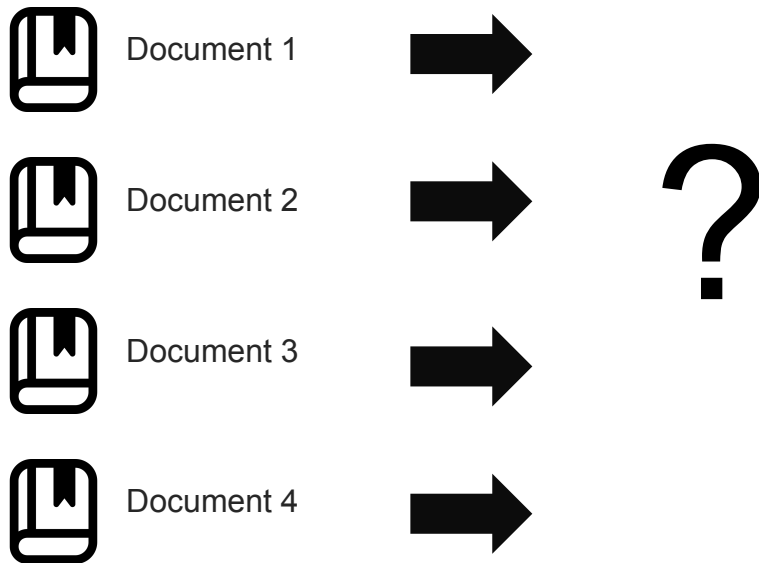
The Challenge: A recruiter searches for "**Python developer with machine learning experience**" but needs to find candidates who wrote:

- "Proficient in Python and ML"
- "Experienced in data science using Python"
- "Strong background in artificial intelligence and Python programming"

Simple keyword matching fails! The same meaning is expressed differently.

NLP = Teaching computers to **understand, interpret, and generate human language**





I want to look for movies about space adventures

Will I enjoy this movie?



Movie 1



Movie 2



Movie 3



| Syntactic models

"Why Is The Rum Always Gone?"



Why	is	the	rum	always	gone	Other words
1	1	1	1	1	1	...

| Syntactic models



Document 1



w1	w2	w3	w4	w5	...	wn
1	0	1	0	1	...	1



Document 2



w1	w2	w3	w4	w5	...	wn
0	3	1	0	1	...	1



Document 3



w1	w2	w3	w4	w5	...	wn
1	0	1	0	1	...	1



Document 4



w1	w2	w3	w4	w5	...	wn
1	0	1	0	1	...	1

Bag of Words

The **Bag-of-Words (BoW) Model** is a feature extraction technique in NLP that converts text into **numerical vectors** by simply **counting the occurrence of each word** in a document. This vector representation allows machine learning algorithms to process text data. Its main limitation is that it **completely ignores word order and grammatical structure**.



I Syntactic models

Simple but they “understand” only syntax.

| Syntactic models

Movie 1: An amazing creature **is running** from a hunter.

['an': 1, 'amazing': 1, 'creature': 1, 'is': 1, 'running': 1, 'from': 1, 'a': 1, 'hunter': 1, ' ': 1]

Movie 2: The hero **ran** through the city to **escape**.

['the': 2, 'hero': 1, 'ran': 1, 'through': 1, 'city': 1, 'to': 1, 'escape': 1, ' ': 1]

Movie 3: **Running** away, the villain **escapes** the **runners**!

['running': 1, 'away': 1, ' ': 1, 'the': 2, 'villain': 1, 'escapes': 1, 'runners!': 1, ' ': 1]

Movie 4: A fun, but slightly **boring**, fantasy film.

['a': 1, 'fun': 1, ' ': 1, 'but': 1, 'slightly': 1, 'boring': 1, 'fantasy': 1, 'film': 1, ' ': 1]

Chapter 2.

Natural Language Processing

- | 1.1. What is NLP?
- | 1.2. Introduction to Search
- | 1.3. Processing text
- | 1.4. Representing text
- | 1.5. Similarity measures
- | 1.6. Next up

Formally speaking we could define search as:

$$F(q,c) = y$$

I.e a function that given a **query** q and a **corpus** c we get a set of **relevant** results y

Then, building a good search system means **optimizing** each part:

- crafting the right **query representation** q ,
- structuring or *embedding* the **corpus** c effectively,
- and designing or learning the best **retrieval function** F

Query: How the user expresses intent, can we make it **clearer**, **richer**, or **smarter**?

Corpus: How the information is **stored** and **represented**, can we make it easier to match, faster to search, or richer in features?

Function: How we **match** and **rank**, can we use better algorithms, learning signals, or hybrid strategies?

Query (q)	Corpus (c)	Retrieval Function (F)
Text representation / embeddings	Raw vs. processed content	Matching strategy (keywords, vectors)
Query expansion / synonyms	Indexing (inverted, vector)	Ranking (TF-IDF, BM25, learned)
Contextualization (history, intent)	Preprocessing / cleaning	Learning & adaptation (ML, feedback loops)
Structured queries / filters	Metadata / structure	Hybrid approaches (lexical + semantic)

Chapter 2.

Natural Language Processing

- | 1.1. What is NLP?
- | 1.2. Introduction to Search
- | 1.3. Processing text
- | 1.4. Representing text
- | 1.5. Similarity measures
- | 1.6. Next up

| Syntactic models

Movie 1: An amazing creature **is running** from a hunter.

['an': 1, 'amazing': 1, 'creature': 1, 'is': 1, 'running': 1, 'from': 1, 'a': 1, 'hunter': 1, ' ': 1]

Movie 2: The hero **ran** through the city to **escape**.

['the': 2, 'hero': 1, 'ran': 1, 'through': 1, 'city': 1, 'to': 1, 'escape': 1, ' ': 1]

Movie 3: **Running** away, the villain **escapes** the **runners**!

['running': 1, 'away': 1, ' ': 1, 'the': 2, 'villain': 1, 'escapes': 1, 'runners!': 1, ' ': 1]

Movie 4: A fun, but slightly **boring**, fantasy film.

['a': 1, 'fun': 1, ' ': 1, 'but': 1, 'slightly': 1, 'boring': 1, 'fantasy': 1, 'film': 1, ' ': 1]

| Syntactic models

Vocabulary

all unique words in a corpus; each word becomes a feature in syntactic models

['an', 'amazing', 'creature', 'is', 'running', 'from', 'a', 'hunter', '.', 'the', 'hero', 'ran', 'through', 'city', 'to', 'escape', 'away', ',', 'villain', 'escapes', 'runners!', '!', 'fun', 'but', 'slightly', 'boring', 'fantasy', 'film']

| Syntactic models

Vocabulary

all unique ~~words~~ tokens in a corpus; each word becomes a feature in syntactic models

['an', 'amazing', 'creature', 'is', 'running', 'from', 'a', 'hunter', '.', 'the', 'hero', 'ran', 'through', 'city', 'to', 'escape', 'away', ',', 'villain', 'escapes', 'runners!', '!', 'fun', 'but', 'slightly', 'boring', 'fantasy', 'film']

| Syntactic models

Vocabulary

all unique words tokens in a corpus; each word becomes a feature in syntactic models

['an', 'amazing', 'creature', 'is', 'running', 'from', 'a', 'hunter', '.', 'the', 'hero', 'ran', 'through', 'city', 'to', 'escape', 'away', ',', 'villain', 'escapes', 'runners!', '!', 'fun', 'but', 'slightly', 'boring', 'fantasy', 'film']

An amazing creature is running from a hunter

[1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]

Natural Language Processing

What are embeddings ?

Embeddings are vectors that represent real-world objects, like **words**, images, or videos, in a form that **computer algorithms** like machine learning models can easily process.

	Feature #1	Feature #2	Feature #3	Feature #4	...
Document #1	1	0	1	0	0
Document #2	0	0	2	0	0
Document #3	0	1	0	0	1
Document #4	0	0	0	1	0
⋮	0	0	1	0	0

DTM

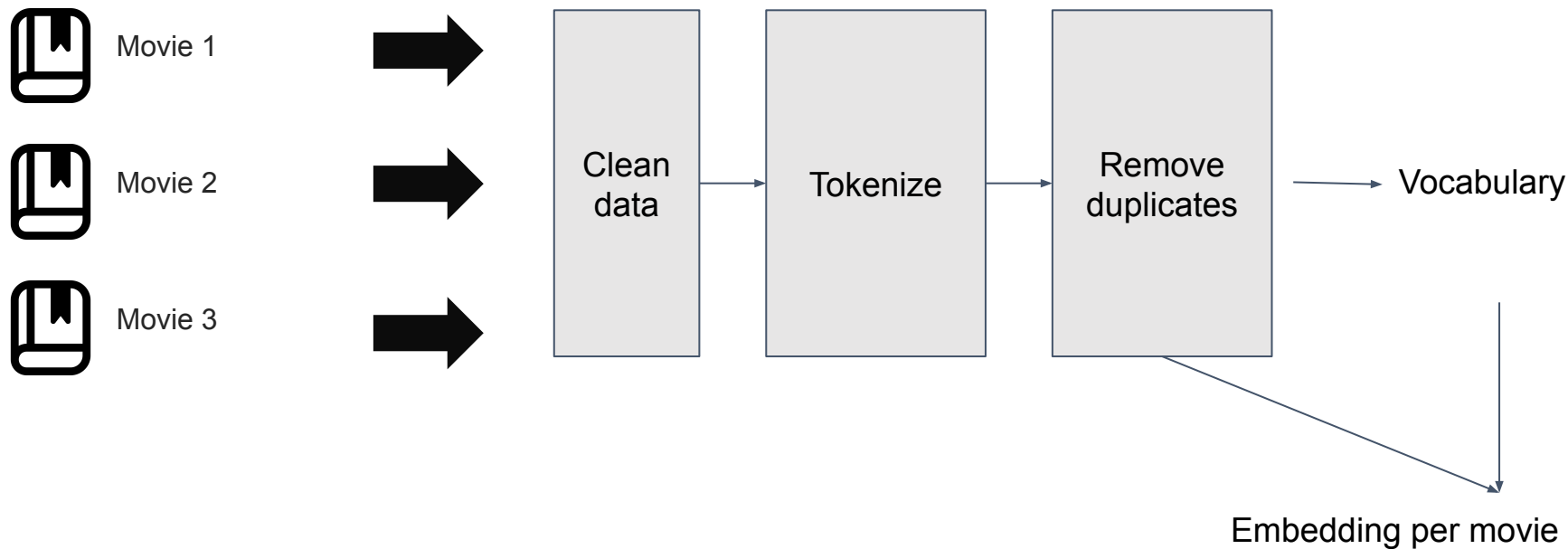
TL;DR it's just a bunch of numbers with meaning for computers

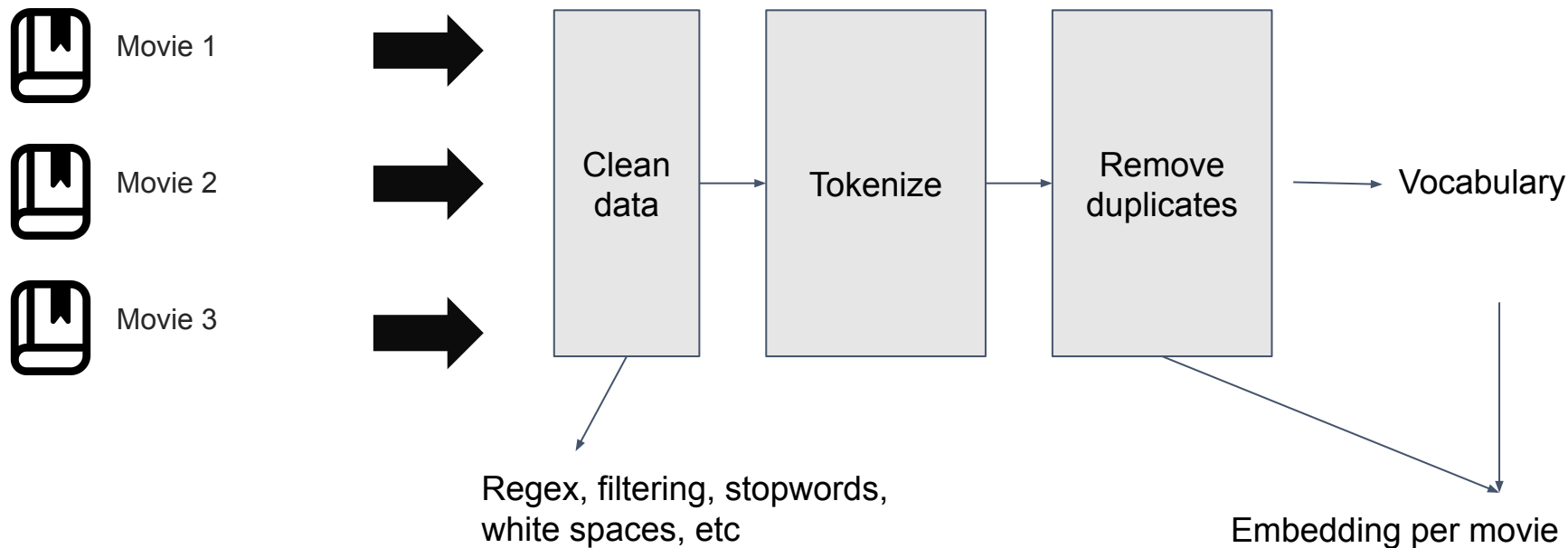
| Syntactic models

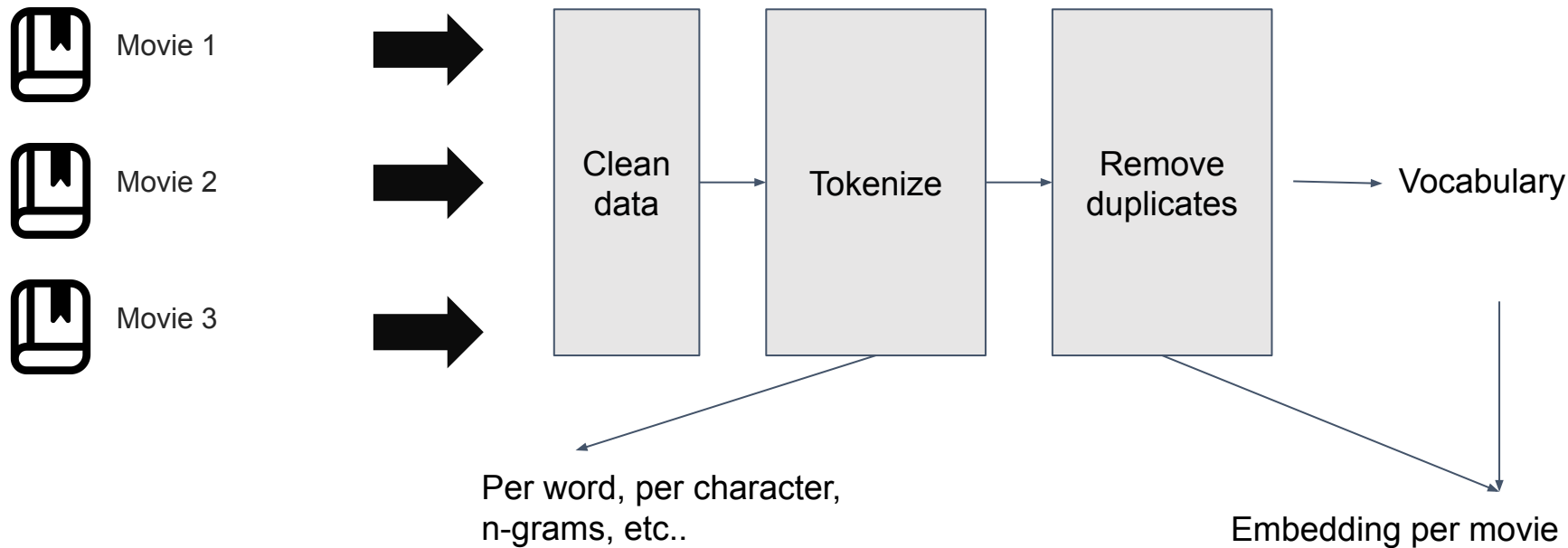
But, are all those **tokens** necessary?

all unique ~~words~~ tokens in a corpus; each word becomes a feature in syntactic models

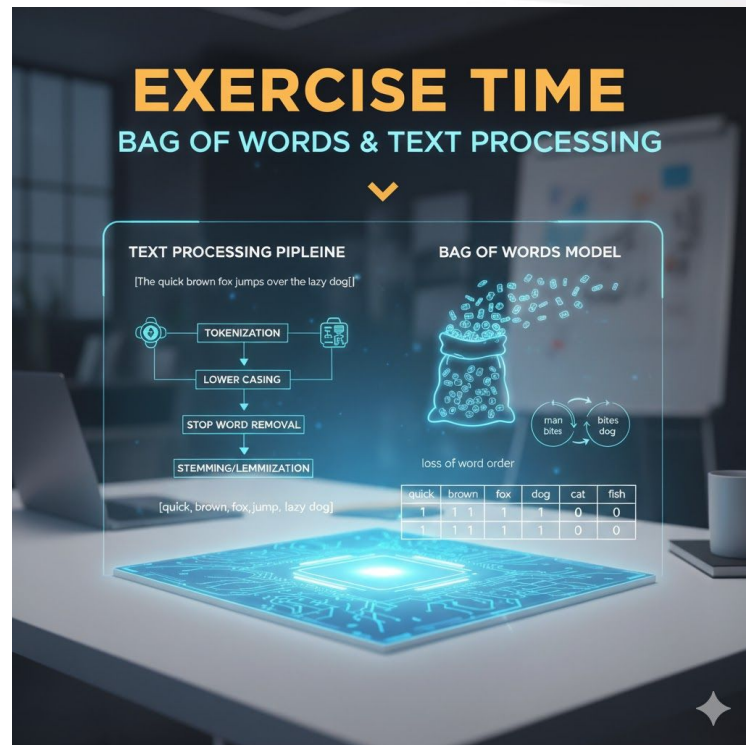
['an', 'amazing', 'creature', 'is', 'running', 'from', 'a', 'hunter', '.', 'the', 'hero', 'ran', 'through', 'city', 'to', 'escape', 'away', ',', 'villain', 'escapes', 'runners!', '!', 'fun', 'but', 'slightly', 'boring', 'fantasy', 'film']







[Let, 's, do ,stuff]



Chapter 2.

Natural Language Processing

- | 1.1. What is NLP?
- | 1.2. Introduction to Search
- | 1.3. Processing text
- | 1.4. Representing text
- | 1.5. Similarity measures
- | 1.6. Next up



Are all documents, movies,... the same?

Is just the frequency of the word enough?

Are all documents, movies,... the same?

Is just the frequency of the word enough?

Doc 1: cats cats cats cats this is about dogs. Dogs like to bark.

Doc 2: Cats like to jump around

Are all documents, movies,... the same?

Is just the frequency of the word enough?

Doc 1: cats cats cats cats this is about dogs. Dogs like to bark.

Doc 2: Cats like to jump around

$V = \{\text{cats, this, is, about, dogs, like, to, bark, jump, around}\}$

Are all documents, movies,.. the same?

Is just the frequency of the word enough?

Doc 1: cats cats cats cats this is about dogs.
Dogs like to bark.

Doc 2: Cats like to jump around

$V = \{\text{cats, this, is, about, dogs, like, to, bark, jump, around}\}$



Are all documents, movies,.. the same?
No, size matters.
Is just the frequency of the word enough?
No. The loudest doesn't always win.



Are all documents, movies,.. the same?
No, size matters.
Is just the frequency of the word enough?
No. The loudest doesn't always win.

A possible solution?

TF-IDF = Term Frequency x Inverse
Document Frequency



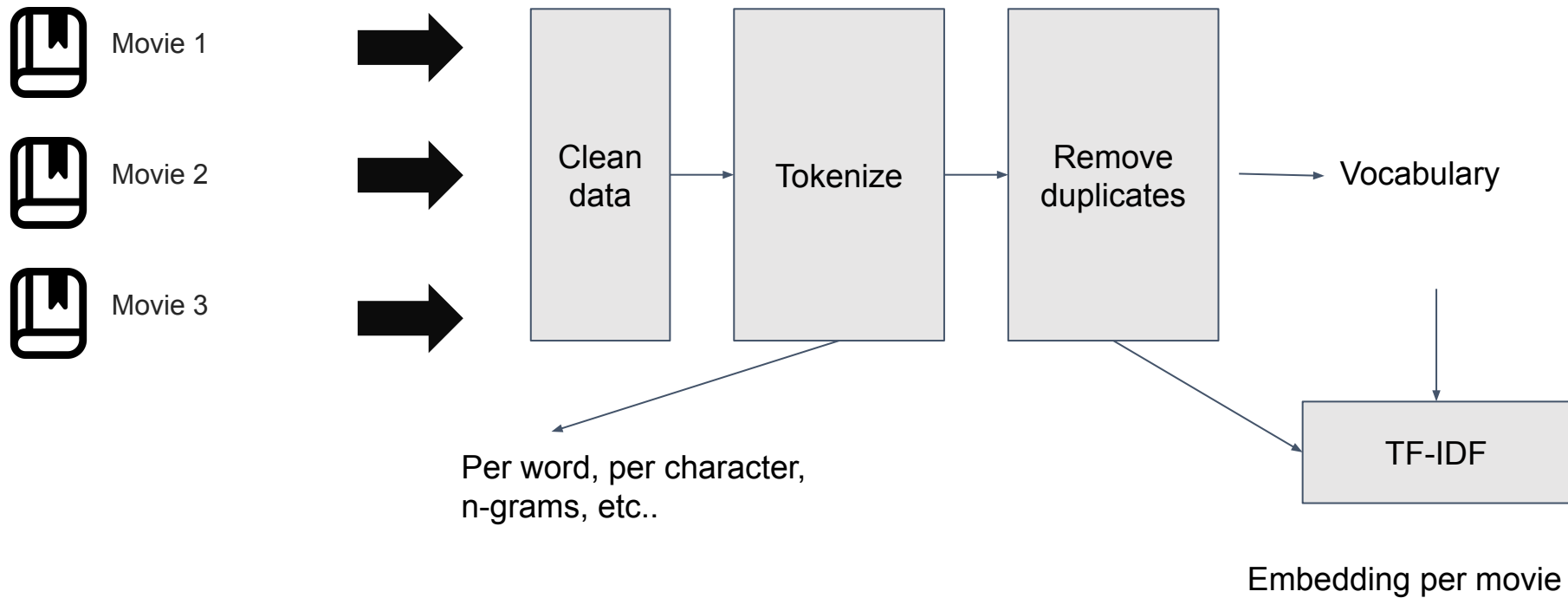
Are all documents, movies,.. the same?
No, size matters.
Is just the frequency of the word enough?
No. The loudest doesn't always win.

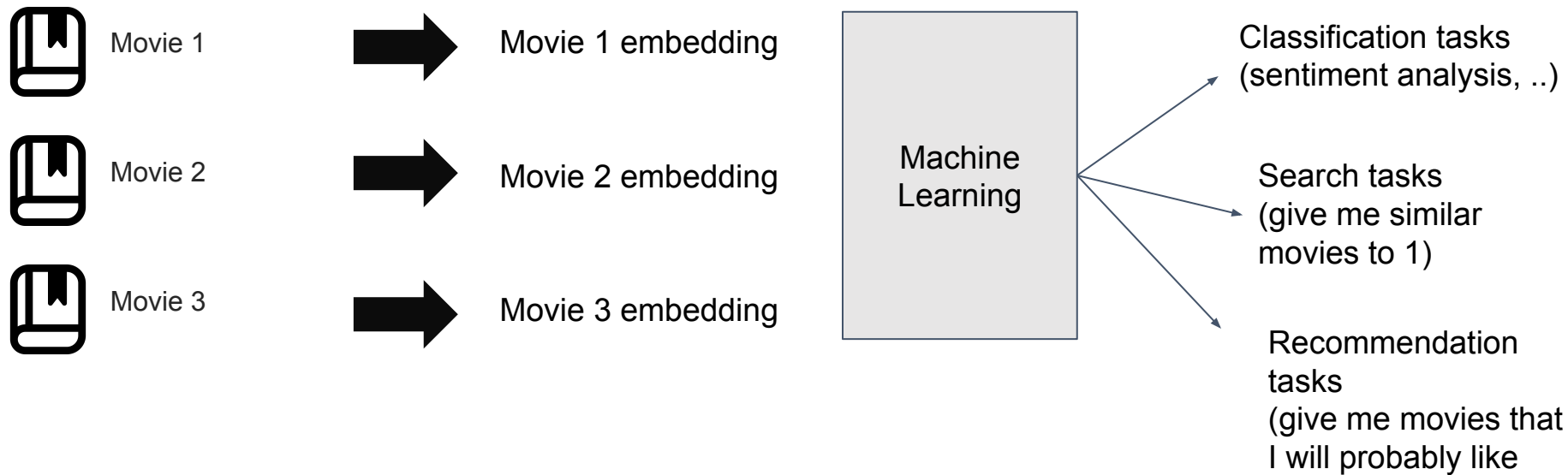
A possible solution?

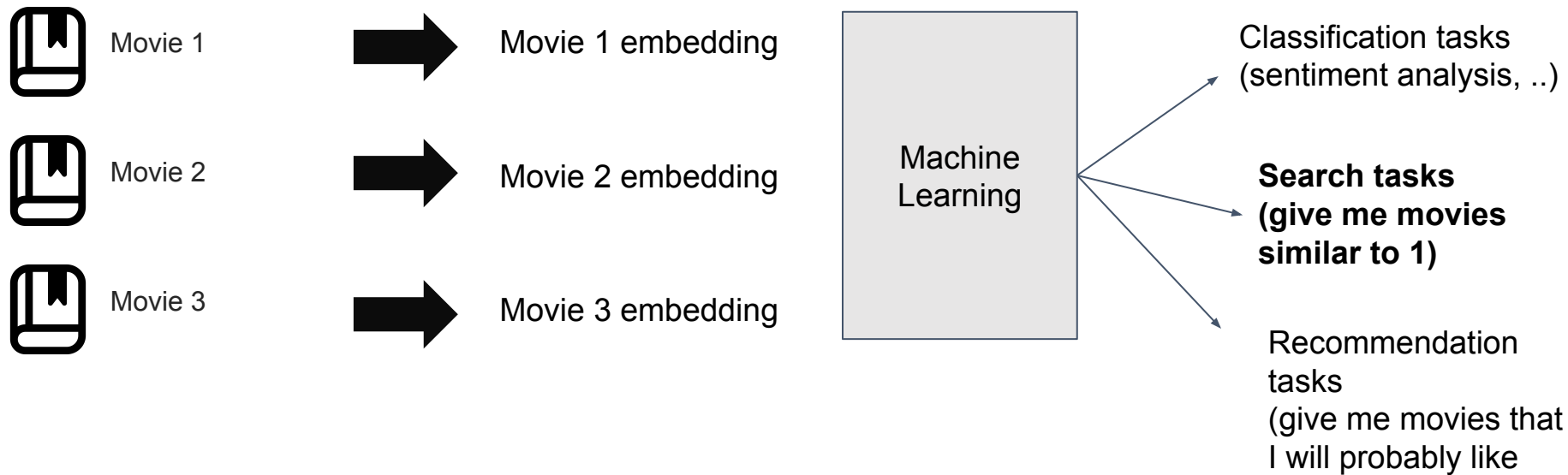
TF-IDF = Term Frequency x Inverse
Document Frequency

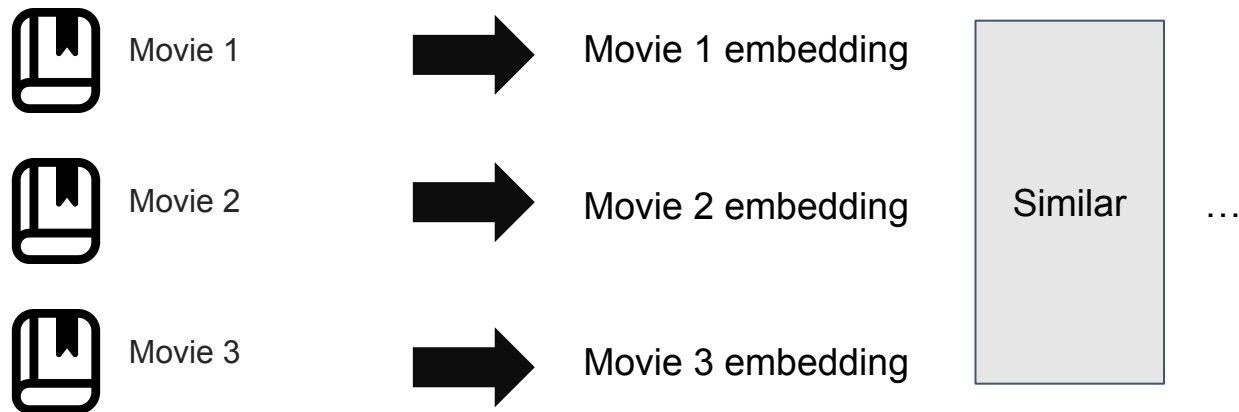
Term Frequency = Bag of Words (the
frequency of each word in the vocabulary)

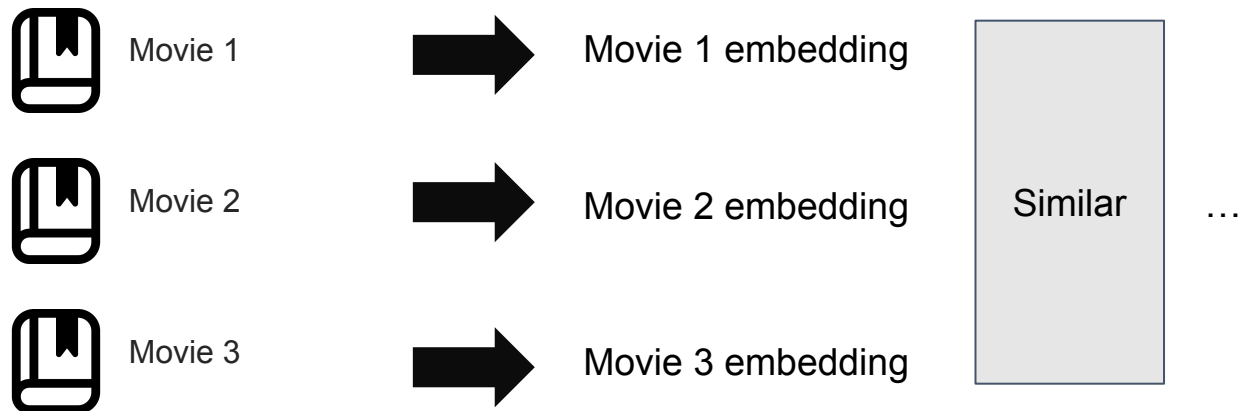
Inverse Document Frequency = How unique
a word is across **all** documents











Embedding = Vector

What are embeddings ?

Embeddings are vectors that represent real-world objects, like **words**, images, or videos, in a form that **computer algorithms** like machine learning models can easily process.

	Feature #1	Feature #2	Feature #3	Feature #4	...
Document #1	1	0	1	0	0
Document #2	0	0	2	0	0
Document #3	0	1	0	0	1
Document #4	0	0	0	1	0
⋮	0	0	1	0	0

DTM

TL;DR it's just a bunch of numbers with meaning for computers

Vector Space Model

Treat each document as a **point in space**, where mathematical tools can be used to measure distances and relationships.

Recall that the TF-IDF process converts a document into a list of weighted scores, one for every unique word in the vocabulary.

Definition: In a **Vector Space Model (VSM)**, a document is represented as a **vector**.

- Each **dimension** of the space corresponds to a unique word in the corpus vocabulary.
- The **value** along each dimension is the calculated TF-IDF weight for that word in the document.

{cats, this, is, around}. This means our VSM is a **10-dimensional space**.

Vector Space Model

Treat each document as a **point in space**, where mathematical tools can be used to **measure distances and relationships**.



Vector Space Model

Treat each document as a **point in space**, where mathematical tools can be used to **measure distances and relationships**.

Some similarities for exact nearest neighbor:

- Cosine Similarity
- Euclidean Similarity
- Jaccard Similarity

Vector Space Model

Treat each document as a **point in space**, where mathematical tools can be used to **measure distances and relationships**.

P.s in the real world with millions of documents, where speed and costs matter, we tend to use Approximate Nearest Neighbors (ANN) algorithms instead of Exact Nearest Neighbors, one of the most famous ones is the Hierarchical Navigable Small Worlds.

[Let, 's, do ,stuff, again]

